

第 10 节 实操任务 2 搭建 5 节点工作流

学习目标: `python workflows/graph.py` 能流式输出 5 个节点的执行日志
目标文件: `workflows/nodes.py` + `workflows/graph.py`

步骤 1: 用 AI 编程工具生成 nodes.py

以下代码可以用 OpenCode、Claude Code、Cursor、Trae 或通义灵码等任意 AI 编程工具生成。

提示词:

```
1 请帮我编写 workflows/nodes.py, 定义 LangGraph 工作流的 5 个节点函数:
2
3 需求:
4 1. 每个节点是纯函数: 接收 KBState, 返回 dict (部分状态更新)
5 2. 节点列表:
6     - collect_node: 调用 GitHub Search API 采集 AI 相关仓库 (urllib.request)
7     - analyze_node: 用 LLM 对每条数据生成中文摘要、标签、评分
8     - organize_node: 过滤低分条目(< 0.6)、按 URL 去重、如有审核反馈则用 LLM 修正
9     - review_node: LLM 四维度评分 (摘要质量/标签准确/分类合理/一致性), iteration
10    - save_node: 将 articles 写入 knowledge/articles/ 目录的 JSON 文件
11 3. 使用 workflows/model_client.py 的 chat() 和 chat_json() 函数
12    - chat(prompt, system=...) 返回 (text, usage)
13    - chat_json(prompt, system=...) 返回 (parsed_json, usage)
14    - accumulate_usage(tracker, usage) 累加 token 统计
15 4. 使用 workflows/state.py 的 KBState
16 5. 每个节点开头打印 [NodeName] 日志
17
18 关键设计点:
19 - review_node 的审核 prompt 要求输出 JSON: {"passed": bool, "overall_score": float}
20 - organize_node 在 iteration > 0 且有 feedback 时, 调用 LLM 做定向修改
21 - save_node 同时更新 index.json 索引文件
```

生成的代码 (参考实现, 节选关键节点):

```
1 """工作流节点定义 - 5 个核心节点
2
3 节点调用链:
4     collect → analyze → organize → review → (conditional) → save
5                                     ↑
6                                     └─ organize (retry) ─┐
7
8 """
9
10 import json
```

```

10 import os
11 from datetime import datetime, timezone
12
13 from workflows.model_client import accumulate_usage, chat, chat_json
14 from workflows.state import KBState
15
16
17 def collect_node(state: KBState) -> dict:
18     """采集节点：调用 GitHub Search API 获取 AI 相关仓库"""
19     import urllib.request
20
21     today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
22     sources: list[dict] = []
23
24     url = "https://api.github.com/search/repositories?q=topic:ai+topic:a
25     headers = {"Accept": "application/vnd.github.v3+json"}
26     token = os.getenv("GITHUB_TOKEN", "")
27     if token:
28         headers["Authorization"] = f"token {token}"
29
30     try:
31         req = urllib.request.Request(url, headers=headers)
32         with urllib.request.urlopen(req, timeout=15) as resp:
33             data = json.loads(resp.read().decode())
34
35         for repo in data.get("items", []):
36             sources.append({
37                 "source": "github",
38                 "title": repo["full_name"],
39                 "url": repo["html_url"],
40                 "description": repo.get("description", ""),
41                 "stars": repo.get("stargazers_count", 0),
42                 "collected_at": datetime.now(timezone.utc).isoformat(),
43             })
44     except Exception as e:
45         sources.append({"source": "github", "title": "[ERROR]", "descrip
46
47     print(f"[Collector] 采集到 {len(sources)} 条原始数据")
48     return {"sources": sources}
49
50
51 def analyze_node(state: KBState) -> dict:
52     """分析节点：用 LLM 对每条数据进行深度分析"""
53     sources = state["sources"]
54     analyses: list[dict] = []
55     tracker = state.get("cost_tracker", {})
56
57     for item in sources:

```

```

58         if item.get("title", "").startswith("[ERROR]"):
59             continue
60
61         prompt = f"""请分析以下技术项目，用 JSON 格式返回：
62 项目名称: {item['title']}
63 描述: {item.get('description', '无描述')}
64
65 返回格式: {{{"summary": "200字中文摘要", "tags": ["标签"], "relevance_score":
66
67         try:
68             result, usage = chat_json(prompt)
69             tracker = accumulate_usage(tracker, usage)
70             analyses.append(**item, **result)
71         except Exception as e:
72             print(f"[Analyzer] 分析失败: {item['title']} - {e}")
73             analyses.append(**item, "summary": f"分析失败: {e}", "relevance_score": 0)
74
75     print(f"[Analyzer] 完成 {len(analyses)} 条分析")
76     return {"analyses": analyses, "cost_tracker": tracker}
77
78
79 def organize_node(state: KBState) -> dict:
80     """整理节点：过滤 + 去重 + 如有反馈则修正"""
81     analyses = state["analyses"]
82     feedback = state.get("review_feedback", "")
83     iteration = state.get("iteration", 0)
84     tracker = state.get("cost_tracker", {})
85
86     # 过滤低分条目
87     qualified = [a for a in analyses if a.get("relevance_score", 0) >= 0]
88
89     # URL 去重
90     seen, unique = set(), []
91     for item in qualified:
92         url = item.get("url", "")
93         if url and url not in seen:
94             seen.add(url)
95             unique.append(item)
96
97     # 如果有审核反馈，用 LLM 修正
98     if feedback and iteration > 0:
99         prompt = f"""你是知识库编辑。请据以下反馈改进条目。
100 反馈: {feedback}
101 条目: {json.dumps(unique, ensure_ascii=False)}
102 返回改进后的 JSON 数组。"""
103
104         try:
105             improved, usage = chat_json(prompt)

```

```

106         tracker = accumulate_usage(tracker, usage)
107         if isinstance(improved, list):
108             unique = improved
109     except Exception:
110         pass
111
112     # 格式化为标准条目
113     today = datetime.now(timezone.utc).strftime("%Y-%m-%d")
114     articles = [
115         {"id": f"{today}-{i:03d}", **{k: item.get(k, "") for k in ["titl
116         for i, item in enumerate(unique)
117     ]
118
119     print(f"[Organizer] 整理出 {len(articles)} 条知识条目 (迭代 {iteration})")
120     return {"articles": articles, "cost_tracker": tracker}
121
122
123 def review_node(state: KBState) -> dict:
124     """审核节点：LLM 四维度评分"""
125     articles = state.get("articles", [])
126     iteration = state.get("iteration", 0)
127     tracker = state.get("cost_tracker", {})
128
129     if not articles:
130         return {"review_passed": True, "review_feedback": "没有条目需要审核"}
131
132     prompt = f"""你是知识库质量审核员。请审核以下条目：
133 {json.dumps(articles[:5], ensure_ascii=False, indent=2)}
134
135 评分维度（1-5分）：摘要质量、标签准确性、分类合理性、整体一致性
136 overall_score >= 3.5 即通过。第 {iteration + 1} 次审核（最多3次）。
137
138 返回 JSON: {"passed": true/false, "overall_score": 4.2, "feedback": "改进
139
140     try:
141         result, usage = chat_json(prompt, system="你是严格但公正的知识库审核员")
142         tracker = accumulate_usage(tracker, usage)
143         passed = result.get("passed", False)
144         feedback = result.get("feedback", "")
145
146         if iteration >= 2:
147             passed = True
148             feedback += "\n[系统] 已达最大审核次数(3次)，强制通过。"
149
150         print(f"[Reviewer] 得分: {result.get('overall_score', '?')}, 通过
151     except Exception as e:
152         passed, feedback = True, f"审核失败: {e}, 自动通过"
153

```

```

154     return {"review_passed": passed, "review_feedback": feedback, "itera
155
156
157 def save_node(state: KBState) -> dict:
158     """保存节点：写入 JSON 文件 + 更新索引"""
159     articles = state.get("articles", [])
160     if not articles:
161         print("[Saver] 没有条目需要保存")
162         return state
163
164     base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
165     articles_dir = os.path.join(base_dir, "knowledge", "articles")
166     os.makedirs(articles_dir, exist_ok=True)
167
168     for article in articles:
169         filepath = os.path.join(articles_dir, f"{article['id']}.json")
170         with open(filepath, "w", encoding="utf-8") as f:
171             json.dump(article, f, ensure_ascii=False, indent=2)
172
173     # 更新索引
174     index_path = os.path.join(articles_dir, "index.json")
175     index = json.load(open(index_path, encoding="utf-8")) if os.path.exi
176     existing_ids = {e["id"] for e in index}
177     for a in articles:
178         if a["id"] not in existing_ids:
179             index.append({"id": a["id"], "title": a.get("title", ""), "c
180     with open(index_path, "w", encoding="utf-8") as f:
181         json.dump(index, f, ensure_ascii=False, indent=2)
182
183     print(f"[Saver] 保存 {len(articles)} 篇文章")
184     return state
185
186 ---

```

步骤 2：用 AI 编程工具生成 graph.py

提示词：

```

1 请帮我编写 workflows/graph.py, 组装 LangGraph 工作流：
2
3 需求：
4 1. 使用 langgraph.graph 的 StateGraph, END
5 2. 导入 workflows/nodes.py 的 5 个节点函数
6 3. 导入 workflows/state.py 的 KBState
7 4. 线性边：collect → analyze → organize → review
8 5. 条件边：review 之后根据 review_passed 分支
9     - True → save → END
10    - False → organize (回到整理节点修正)
11 6. 入口点：collect

```

```

12 7. build_graph() 函数返回编译后的 app
13 8. if __name__ == "__main__" 流式执行并打印每个节点的关键输出
14
15 注意：使用真实的 LangGraph API：
16 - StateGraph(KBState)
17 - graph.add_node("name", function)
18 - graph.add_edge("a", "b")
19 - graph.add_conditional_edges("source", router_fn, {"key": "target"})
20 - graph.set_entry_point("name")
21 - graph.add_edge("save", END)
22 - graph.compile()

```

生成的代码（参考实现）：

```

1  """LangGraph  workflow图定义 - 采集-分析-审核流水线
2
3   workflow拓扑:
4      collect → analyze → organize → review → save (通过)
5                                     ↑
6      └────────────────────────────────┘ (未通过)
7
8  """
9  from langgraph.graph import END, StateGraph
10
11  from workflows.nodes import (
12      analyze_node, collect_node, organize_node, review_node, save_node,
13  )
14  from workflows.state import KBState
15
16
17  def should_continue(state: KBState) -> str:
18      """条件路由：审核通过 → save, 未通过 → organize"""
19      if state.get("review_passed", False):
20          return "save"
21      return "organize"
22
23
24  def build_graph():
25      """构建知识库 V3  workflow"""
26      graph = StateGraph(KBState)
27
28      # 添加节点
29      graph.add_node("collect", collect_node)
30      graph.add_node("analyze", analyze_node)
31      graph.add_node("organize", organize_node)
32      graph.add_node("review", review_node)
33      graph.add_node("save", save_node)
34
35      # 线性边

```

```

36     graph.add_edge("collect", "analyze")
37     graph.add_edge("analyze", "organize")
38     graph.add_edge("organize", "review")
39
40     # 条件边：审核循环
41     graph.add_conditional_edges(
42         "review", should_continue, {"save": "save", "organize": "organiz
43     )
44
45     # 终点
46     graph.add_edge("save", END)
47
48     # 入口
49     graph.set_entry_point("collect")
50
51     return graph
52
53
54 app = build_graph().compile()
55
56
57 if __name__ == "__main__":
58     print("=" * 60)
59     print("AI 知识库 - LangGraph  workflow启动")
60     print("=" * 60)
61
62     initial_state: KBState = {
63         "sources": [], "analyses": [], "articles": [],
64         "review_feedback": "", "review_passed": False,
65         "iteration": 0, "cost_tracker": {},
66     }
67
68     for event in app.stream(initial_state):
69         node_name = list(event.keys())[0]
70         print(f"\n--- [{node_name}] 完成 ---")
71
72     print("\n" + "=" * 60)
73     print("workflow执行完毕")
74
75 ---

```

步骤 3: 理解代码

请解释 workflows/graph.py 的设计:

1. `add_conditional_edges` 的三个参数分别是什么意思?
2. `should_continue` 返回的字符串和第三个参数的字典有什么关系?
3. `review` 未通过时回到 `organize`, `organize` 怎么知道要修正?

步骤 4: 运行测试

```
1 cd ~/ai-knowledge-base
2 python workflows/graph.py
```

期望输出:

```
1 =====
2 AI 知识库 - LangGraph 工作流启动
3 =====
4 [Collector] 采集到 10 条原始数据
5
6 --- [collect] 完成 ---
7 [Analyzer] 完成 8 条分析
8
9 --- [analyze] 完成 ---
10 [Organizer] 整理出 6 条知识条目 (迭代 0)
11
12 --- [organize] 完成 ---
13 [Reviewer] 得分: 3.8, 通过: True (迭代 1/3)
14
15 --- [review] 完成 ---
16 [Saver] 保存 6 篇文章
17
18 --- [save] 完成 ---
19 =====
20 工作流执行完毕
21
22 ---
```

步骤 5: 提交到 Git

```
1 git add workflows/nodes.py workflows/graph.py
2 git commit -m "feat: add 5-node LangGraph workflow with review loop"
3
4 ---
```

完成! 5 个节点的 LangGraph 工作流已经能跑了。下一步加条件边测试审核循环。